Catálogo de funciones

Funciones generales y repetidas de tus librerías, con el código tal como lo escribiste.

Sección	Funciones
1. Bits y archivos binarios	`imprimirhex, imprimirbin, leerArchivo`
2. Grilla (nodos up / down / left / right)	`imprimirGrilla, imprimirGrilla2, freeMem, imprimirPos, printPath, contadorFila, difFila, borrarNodo, recortarGrilla, ordenarGrilla, imprimirnextGrilla`
3. Lista simple / circular (nodo)	`crear_nodo, toggleCirc, freeLista, ins_first, del_last, imprimirlista, armarLista`
4. Lista doblemente enlazada (dnode_t / NodoDoble)	`crear_nodo, funcionb, imprimirlistasolo, imprimirLista, toggleCirc, freeLista`
5. Lista circular con máscaras (Node)	`imprimirGrilla, imprimirGrillaM, limpiarGrilla, freeLista`
6. Grilla tipo buscaminas (qnode_t, Ejercicio 2)	`imp_grid, flag, cumple, mine`
7. Árbol binario de búsqueda	`crearNodo, insOrdenado, printTree`

Cada función indica el archivo del que fue tomada. Cuando una función se repite con pequeñas variaciones (tipo de dato o firma), se muestran las variantes distintas.

1. Bits y archivos binarios

Impresión en hexadecimal/binario y lectura de un archivo binario de registros aplicando operaciones SET/RESET sobre un entero. Patrón de archivos: fopen / while (fread) / procesar.

`imprimirhex`

funciones.c — Imprime un uint16_t en hexa.

```
void imprimirhex(uint16_t dato)
{
  char digitos[] = "0123456789ABCDEF";

  printf("0x");
  for (int i = 16; i > 0; i--)
  {
    printf("%c", digitos[(dato >> (i - 1) * 4) & 0xF]);
  }
  printf("\n");
}
```

`imprimirbin`

funciones.c — Imprime un uint64_t en bloques de 4 bits.

```
void imprimirbin(uint64_t dato)
{
  for (int i = 64; i > 0; i--)
  {
    if (i % 4 == 0)
    {
      printf(" ");
    }
    printf("%ld", (dato >> i - 1) & 1);
  }
  printf("\n");
}
```

`leerArchivo`

funciones.c — Lee M13.bin y arma el canvas con SET / RESET de bits.

```
uint16_t leerArchivo()
{
  FILE *list = fopen("M13.bin", "rb");
  M13_t templeido;
  int16_t canvas = 0, editor = 0;

  if (list == NULL)
  {
    perror("No se pudo abrir el archivo");
    return 1;
  }

  while (fread(&templeido, sizeof(M13_t), 1, list))
  {
    if (templeido.act == SET)
    {
      editor = 1 << templeido.nrBits - 1;
      canvas = canvas | editor;
    }
    else if (templeido.act == RESET)
    {
      editor = 1 << templeido.nrBits - 1;
      canvas = canvas & ~editor;
    }
    else
    {
      printf("no se ha podido interpretar la consigna");
    }
  }

  return canvas;
}
```

2. Grilla (nodos up / down / left / right)

Grilla enlazada en cuatro direcciones. Se recorre fila por fila con first->down y, dentro de cada fila, con ->right. Las versiones aparecen repetidas con distinto tipo de dato (%d / %c).

`imprimirGrilla`

funciones.c — Imprime la grilla (dato int). Idéntica en funciones_0121e7.c; en funcs_abfa87.c usa %c.

```
void imprimirGrilla(qnode_t *first)
{
  qnode_t *aux = first;
  qnode_t *aux2 = first;

  while (aux != NULL)
  {
    while (aux != NULL)
    {
      printf("%d | ", aux->valor);
      aux = aux->right;
    }
    printf("\n");
    aux2 = aux2->down;
    aux = aux2;
  }
}
```

`imprimirGrilla`

funcs_abfa87.c — Variante con dato char.

```
void imprimirGrilla(NodoGrilla *first)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;

  while (aux != NULL)
  {
    while (aux != NULL)
    {
      printf("%c ", aux->dato);
      aux = aux->right;
    }
    printf("\n");
    aux2 = aux2->down;
    aux = aux2;
  }
}
```

imprimirGrilla2

funciones.c — Imprime y además completa conexiones right faltantes, contando nodos por fila.

```
void imprimirGrilla2(qnode_t *first)
{
  qnode_t *aux = first;
  qnode_t *aux2 = first;
  int count = 0;

  while (aux != NULL)
  {
    while (aux != NULL)
    {
      printf(" %d ", aux->valor);
      if ((aux->right == NULL) && (aux->down != NULL) && (aux->down->right != NULL))
      {
        aux->right = aux->down->right->up;
      }
      else if ((aux->right == NULL) && (aux->up != NULL) && (aux->up->right != NULL))
      {
        aux->right = aux->up->right->down;
      }
      aux = aux->right;
      count++;
    }
    printf("\t %d\n", count);
    count = 0;
    aux2 = aux2->down;
    aux = aux2;
  }
}
```

freeMem

funciones.c — Libera la grilla. Versión de funciones.c (guarda el siguiente de la fila antes de free).

```
void freeMem(qnode_t *first)
{
  qnode_t *aux = first;
  qnode_t *aux2 = first;
  qnode_t *aux3 = first;
  printf("\nLIBERANDO MEMORIA \n");

  while (aux != NULL)
  {
    aux2 = aux2->down;
    while (aux != NULL)
    {
      aux3 = aux->right;
      printf("%d | ", aux->valor);
      free(aux);
      aux = aux3;
    }
    printf("\n");
    aux = aux2;
  }

  return;
}
```

freeMem

funcs.c — Variante con dato char, también en funcs_1.c y funcs_abfa87.c.

```
void freeMem(NodoGrilla *first)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  printf("\n");

  while (aux != NULL)
  {
    while (aux != NULL)
    {
      aux3 = aux->right;
      printf("%c | ", aux->dato);
      free(aux);
      aux = aux3;
    }
    printf("\n");
    aux2 = aux2->down;
    aux = aux2;
  }

  return;
}
```

imprimirPos

funcs.c — Busca un nodo y devuelve su posición en una estructura Posicion.

```
void imprimirPos(NodoGrilla *first, NodoGrilla *start, Posicion *pos)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  int posx = 0, posy = -1;

  // printf("\n");
  while (aux != NULL)
  {
    posx = 0;
    posy++;
    while (aux != NULL)
    {
      aux3 = aux->right;
      // printf("%c | ", aux->dato);
      if (aux->dato == start->dato)
      {
        pos->columna = posx;
        pos->fila = posy;
        return;
      }
      aux = aux3;
      posx++;
    }
    // printf("\n");
    aux2 = aux2->down;
    aux = aux2;
  }
}
```

imprimirPos

funcs_1.c — Variante que devuelve la posición por punteros a int.

```
void imprimirPos(NodoGrilla *first, NodoGrilla *start, int size, int distancia, int *posFinalx, int *posFinaly)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  int posx = 0, posy = -1;

  // printf("\n");
  while (aux != NULL)
  {
    posx = 0;
    posy++;
    while (aux != NULL)
    {
      aux3 = aux->right;
      // printf("%c | ", aux->dato);
      if (aux->dato == start->dato)
      {
        *posFinalx = posx;
        *posFinaly = posy;
        return;
      }
      aux = aux3;
      posx++;
    }
    // printf("\n");
    aux2 = aux2->down;
    aux = aux2;
  }
}
```

printPath

funcs.c — Camino entre dos posiciones (Posicion), imprimiendo con flechas.*

```
void printPath(NodoGrilla *first, Posicion *posStart, Posicion *posFinal)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  int posx = 0, posy = 0;

  while (posx != posStart->columna)
  {
    if (posx < posStart->columna)
    {
      aux = aux->right;
      posx++;
    }
    if (posx > posStart->columna)
    {
      aux = aux->left;
      posx--;
    }
  }

  while (posy != posStart->fila)
  {
    if (posy < posStart->fila)
    {
      aux = aux->down;
      posy++;
    }
    if (posy > posStart->fila)
    {
      aux = aux->up;
      posy--;
    }
  }

  printf("\n");

  printf(" %c", aux->dato);
  while (posx != posFinal->columna)
  {
    if (posx < posFinal->columna)
    {
      aux = aux->right;
      printf(" > %c", aux->dato);
      posx++;
    }
    if (posx > posFinal->columna)
    {
      aux = aux->left;
      printf(" < %c", aux->dato);
      posx--;
    }
  }

  while (posy != posFinal->fila)
  {
    if (posy < posFinal->fila)
    {
      aux = aux->down;
      printf(" ˇ %c", aux->dato);
      posy++;
    }
    if (posy > posFinal->fila)
    {
      aux = aux->up;
      printf(" ^ %c", aux->dato);
      posy--;
    }
  }
}
```

printPath

funcs_1.c — Variante con posiciones como int.

```
void printPath(NodoGrilla *first, int posStartx, int posStarty, int posFinalx, int posFinaly)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  int posx = 0, posy = 0;

  while (posx != posStartx)
  {
    if (posx < posStartx)
    {
      aux = aux->right;
      posx++;
    }
    if (posx > posStartx)
    {
      aux = aux->left;
      posx--;
    }
  }

  while (posy != posStarty)
  {
    if (posy < posStarty)
    {
      aux = aux->down;
      posy++;
    }
    if (posy > posStarty)
    {
      aux = aux->up;
      posy--;
    }
  }

  printf("\n");

  printf(" %c", aux->dato);
  while (posx != posFinalx)
  {
    if (posx < posFinalx)
    {
      aux = aux->right;
      printf(" > %c", aux->dato);
      posx++;
    }
    if (posx > posFinalx)
    {
      aux = aux->left;
      printf(" < %c", aux->dato);
      posx--;
    }
  }

  while (posy != posFinaly)
  {
    if (posy < posFinaly)
    {
      aux = aux->down;
      printf(" / %c", aux->dato);
      posy++;
    }
    if (posy > posFinaly)
    {
      aux = aux->up;
      printf(" ^ %c", aux->dato);
      posy--;
    }
  }
}
```

contadorFila

funcs_abfa87.c — Cuenta los nodos de una fila.

```
int contadorFila(NodoGrilla *first, int fila)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  int auxFila = 0, cont = 0;

  // printf("\nContando Fila: %d \n", fila);
  while (aux != NULL)
  {
    if (auxFila == fila)
    {
      while (aux != NULL)
      {
        // printf("%c ", aux->dato);
        aux = aux->right;
        cont++;
      }
      // printf("\n");
      break;
    }
    aux2 = aux2->down;
    aux = aux2;
    auxFila++;
  }

  return cont;
}
```

difFila

funcs_abfa87.c — Diferencia entre una fila y la fila más corta.

```
int difFila(NodoGrilla *first, int fila)
{
  int cantmin = __INT_MAX__, temp = 0, cont = 0, dif = 0;
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;

  while (aux != NULL)
  {
    temp = contadorFila(first, cont);
    if (temp < cantmin)
    {
      cantmin = temp;
    }
    aux2 = aux2->down;
    aux = aux2;
    cont++;
  }

  temp = contadorFila(first, fila);
  dif = temp - cantmin;
  // printf("\n %d \n ", dif);
  return dif;
}
```

borrarNodo

funcs_abfa87.c — Corta las conexiones de un nodo con sus vecinos.

```
void borrarNodo(NodoGrilla *aux)
{
  if (aux->left != NULL)
  {
    aux->left->right = NULL;
  }
  if (aux->right != NULL)
  {
    aux->right->left = NULL;
  }
  if (aux->up != NULL)
  {
    aux->up->down = NULL;
  }
  if (aux->down != NULL)
  {
    aux->down->up = NULL;
  }
}
```

recortarGrilla

funcs_abfa87.c — Remueve los últimos N nodos de una fila.

```
void recortarGrilla(NodoGrilla *first, int fila, int cant)
{
  NodoGrilla *aux = first;
  NodoGrilla *aux2 = first;
  NodoGrilla *aux3 = first;
  int auxFila = 0, longFila = 0, cont = 0;

  longFila = contadorFila(first, fila);

  printf("\nBorrando Fila: %d \n", fila);
  while (aux != NULL)
  {
    if (auxFila == fila)
    {
      while (cont < longFila - cant)
      {
        printf("%c ", aux->dato);
        aux = aux->right;
        cont++;
      }
      while (cont < longFila)
      {
        printf(" -%c ", aux->dato);
        aux3 = aux->right;
        borrarNodo(aux);
        aux = aux3;
        cont++;
      }
      printf("\n---------Listo---------\n");
      break;
    }
    aux2 = aux2->down;
    aux = aux2;
    auxFila++;
  }
}
```

ordenarGrilla

funciones_0121e7.c — Ordena la grilla por columnas (burbujeo vertical).

```
void ordenarGrilla(NodoGrilla *first)
{
    NodoGrilla *aux = first;
    NodoGrilla *aux2 = first;
    int auxvalor = 58;
    int cambios = 1;

    while (cambios != 0)
    {
        cambios = 0;
        aux = first;
        aux2 = first;
        printf("\n\n");
        while (aux != NULL)
        {
            while (aux != NULL)
            {
                if (aux->down != NULL)
                {
                    if (aux->dato > aux->down->dato)
                    {
                        aux->down->nextDato = aux->dato;
                        aux->dato = aux->down->dato;
                        aux->down->dato = aux->down->nextDato;
                        cambios = 1;
                    }
                }
                printf("%d | ", aux->dato);
                aux = aux->down;
            }
            printf("\n");
            aux2 = aux2->right;
            aux = aux2;
        }
    }
}
```

imprimirnextGrilla

funciones_0121e7.c — Imprime el campo nextDato de cada nodo.

```
void imprimirnextGrilla(NodoGrilla *first)
{
    NodoGrilla *aux = first;
    NodoGrilla *aux2 = first;

    while (aux != NULL)
    {
        while (aux != NULL)
        {
            printf("%d | ", aux->nextDato);
            aux = aux->right;
        }
        printf("\n");
        aux2 = aux2->down;
        aux = aux2;
    }
}
```

3. Lista simple / circular (nodo)

Lista enlazada simple que puede alternarse a circular. Es la familia más repetida: crear_nodo, toggleCirc, freeLista e imprimirlista con el truco aux / aux2 para detectar el cierre del círculo.

`crear_nodo`

funciones_1.c — Reserva un nodo con dato y next = NULL.

```
nodo *crear_nodo(int num)
{
  nodo *nodoaux = (nodo *)malloc(sizeof(nodo));
  if (nodoaux == NULL)
  {
    printf("No se pudo crear el nodo");
    return NULL;
  }
  nodoaux->dato = num;
  nodoaux->next = NULL;
  return nodoaux;
}
```

`toggleCirc`

funciones_1.c — Alterna la lista entre circular y no circular.

```
void toggleCirc(nodo **first)
{
  nodo *aux = *first;
  nodo *aux2 = NULL;

  while (1)
  {
    aux2 = aux;
    if (aux->next == NULL)
    {
      printf("\n La lista no es circular, La hago circular\n");
      aux->next = *first;
      break;
    }
    if (aux2->next == *first)
    {
      printf("\n La lista es circular, la corto \n");
      aux->next = NULL;
      break;
    }
    aux = aux->next;
  }
  printf("\n");
}
```

`freeLista`

funciones_1.c — Libera la lista (simple o circular).

```
void freeLista(nodo *first)
{
  nodo *aux = first;
  nodo *aux2 = NULL;

  while ((aux2 != first) && (aux != NULL))
  {
    aux2 = aux->next;
    free(aux);
    aux = aux2;
  }
  printf("Memoria Liberada \n");
}
```

ins_first

funciones_1.c — Inserta al principio, respetando circularidad.

```
void *ins_first(nodo **first, int num)
{
  nodo *aux = (*first);
  nodo *nuevoNodo = crear_nodo(num);
  nodo *aux2 = NULL;

  while ((aux != NULL))
  {
    aux2 = aux->next;
    if (aux2 == (*first))
    {
      aux->next = nuevoNodo;
      break;
    }
    aux = aux2;
  }

  nuevoNodo->next = (*first);
  (*first) = nuevoNodo;

  // de esta forma si first es null nos chupa un huevo
  return 0;
}
```

del_last

funciones_1.c — Elimina el último nodo.

```
void del_last(nodo **first)
{
  // no queremos que nos devuelva nada porque el vector que apunta al ultimo nodo no nos importa

  // DECLARO UN AUXILIAR PARA RECORRER LA LISTA
  nodo *aux = *first;

  // VERIFICO QUE HAYA UNA CANTIDAD SUFICIENTE DE NODOS
  if (((*first)->next == NULL) || ((*first)->next == (*first)))
  {
    free(*first);
    *first = NULL;
  }
  nodo *aux2 = aux->next;
  // RECORRO LA LISTA
  while ((aux->next->next != NULL))
  {
    // comparo con next para pararme en el ante-ultimo, porque necesito que quede apuntando a null
    // printf("%d -> ", aux->dato);
    aux = aux->next;
    if ((aux->next->next == (*first)))
    {
      aux2 = aux->next;
      free(aux2);
      aux->next = (*first);
      return;
    }
  }
  aux2 = aux->next;
  free(aux2);
  aux->next = NULL;
  return;
}
```

imprimirlista

funciones_1.c — Imprime la lista (corta al volver al inicio).

```
void imprimirlista(nodo *first)
{
  nodo *aux = first;
  nodo *aux2 = NULL;

  while ((aux2 != first) && (aux != NULL))
  {
    printf("%d -> ", aux->dato);
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");
}
```

armarLista

funciones_1.c — Lee M12.dat y ejecuta operaciones según la máscara.

```
void armarLista()
{
  FILE *list = fopen("M12.dat", "rb");
  M12_t templeido;
  nodo *first = NULL;

  while (fread(&templeido, sizeof(M12_t), 1, list))
  {
    if (templeido.mask == 10)
    {
      printf("%b - %d - 10\n", templeido.mask, templeido.mask);
      toggleCirc(&first);
      imprimirlista(first);
      printf("\n");
    }
    else if (templeido.mask == 11)
    {
      printf("%b - %d - 11\n", templeido.mask, templeido.mask);
      freeLista(first);
      imprimirlista(first);
      printf("\n");
    }
    else if ((templeido.mask & 3) == 2)
    {
      printf("%b - %d - 2\n", templeido.mask, templeido.mask);
      ins_first(&first, templeido.dato);
      imprimirlista(first);
      printf("\n");
    }
    else if ((templeido.mask & 3) == 3)
    {
      printf("%b - %d - 3\n", templeido.mask, templeido.mask);
      del_last(&first);
      imprimirlista(first);
      printf("\n");
      // freeLista(first);
    }
    else
    {
      printf("no se ha podido interpretar la consigna");
    }
  }
}
```

4. Lista doblemente enlazada (dnode_t / NodoDoble)

Versiones para lista doble (prev / next): creación, inserción ordenada, impresión, toggle circular y liberación.

`crear_nodo`

funcs.c — Crea NodoDoble con next y prev en NULL.

```
NodoDoble *crear_nodo(int num)
{
  NodoDoble *nodo = (NodoDoble *)malloc(sizeof(NodoDoble));
  if (nodo == NULL)
  {
    printf("No se pudo crear el nodo");
    return NULL;
  }
  nodo->numero = num;
  nodo->next = NULL;
  nodo->prev = NULL;

  return nodo;
}
```

funcionb

funcs.c — Pasa números de un vector a una lista doble ordenada (de mayor a menor).

```
void funcionb(customElement *vector, int tamanio, NodoDoble **first, NodoDoble **last)
{

  int cantdel = 0;
  NodoDoble *nuevoNodo;
  NodoDoble *aux = *first;
  customElement *chusma;

  for (int i = 0; i < tamanio - cantdel; i++)
  {
    aux = *first;
    chusma = &vector[i];
    if (vector[i].esNro)
    {
      nuevoNodo = crear_nodo((vector[i].dato.numero));
      if (cantdel == 0)
      {
        *first = nuevoNodo;
        *last = nuevoNodo;
      }
      else
      {
        if (aux->next == NULL)
        {
          *last = nuevoNodo;
        }
        else
        {
          while (aux->next->numero > nuevoNodo->numero)
          {
            aux = aux->next;
            if (aux->next == NULL)
            {
              *last = nuevoNodo;
              break;
            }
          }
        }
        if (aux->next != NULL)
        {
          aux->next->prev = nuevoNodo;
        }
        nuevoNodo->next = aux->next;
        aux->next = nuevoNodo;
        nuevoNodo->prev = aux;
      }
      cantdel++;
      printf("%d : %d \n", i, vector[i].dato.numero);
      if (i + 1 < tamanio)
      {
        vector[i] = vector[i + cantdel];
        i--;
      }
    }
    else
    {
      printf("%d : %c \n", i, vector[i].dato.caracter);
    }
  }
}
```

imprimirlistasolo

funciones.c — Imprime una lista doble.

```
void imprimirlistasolo(dnode_t *first)
{
  dnode_t *aux = first;
  dnode_t *aux2 = NULL;

  while ((aux2 != first) && (aux != NULL))
  {
    printf("%d -> ", aux->dato);
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");
}
```

imprimirLista

funciones.c — Imprime desde el medio, detectando primero y último.

```
void imprimirLista(dnode_t **middle, dnode_t **first, dnode_t **last)
{
  dnode_t *aux = *middle;
  dnode_t *aux2 = aux->next; // por si es circular
  dnode_t *auxprev = *middle;
  dnode_t *auxnext = *middle;

  int firstFound = 0, lastFound = 0;

  printf("\n");
  while (((firstFound == 0) || (lastFound == 0)) && (aux2 != *middle))
  {
    if (auxprev->prev == NULL)
    {
      printf("\nSe encontro el principio\n");
      if (firstFound == 0 && lastFound == 0)
      {
        auxprev->prev = (*middle);
        (*middle)->next->prev = (*middle)->prev;
        (*middle)->prev->next = (*middle)->next;
        (*middle)->next = auxprev;
        (*middle)->prev = NULL;
        auxprev = *middle;
      }
      *first = auxprev;
      auxprev = auxnext;
      firstFound = 1;
    }
    if (auxnext->next == NULL)
    {
      printf("\nSe encontro el final\n");
      *last = auxnext;
      if (firstFound == 0 && lastFound == 0)
      {
        auxnext->next = (*middle);
        (*middle)->next->prev = (*middle)->prev;
        (*middle)->prev->next = (*middle)->next;
        (*middle)->next = NULL;
        (*middle)->prev = auxnext;
        auxnext = *middle;
      }
      *last = auxnext;
      auxnext = auxprev;
      lastFound = 1;
    }
    auxprev = auxprev->prev;
    auxnext = auxnext->next;
    aux2 = auxnext;
  }

  printf("\n");
}
```

toggleCirc

funciones.c — Versión para lista doble (ajusta también prev).

```
void toggleCirc(dnode_t **first)
{
  dnode_t *aux = *first;
  dnode_t *aux2 = aux->next;

  while (1)
  {
    if (aux->next == NULL)
    {
      printf("\n La lista no es circular, La hago circular\n");
      aux->next = *first;
      (*first)->prev = aux;
      break;
    }
    if (aux2 == *first)
    {
      printf("\n La lista es circular, la corto \n");
      (*first)->prev->next = NULL;
      (*first)->prev = NULL;
      break;
    }
    aux = aux->next;
    aux2 = aux;
  }

  printf("\n");
}
```

freeLista

funciones.c — Libera una lista doble.

```
void freeLista(dnode_t *first)
{
  dnode_t *aux = first;
  dnode_t *aux2 = NULL;

  while ((aux2 != first) && (aux != NULL))
  {
    aux2 = aux->next;
    free(aux);
    aux = aux2;
  }
  printf("Memoria Liberada \n");
}
```

5. Lista circular con máscaras (Node)

Lista circular de enteros donde se marcan nodos con (dato & 3) == 3 y se limpian los de posición par.

`imprimirGrilla`

funcs_1.c — Imprime la lista en binario y en decimal.

```
void imprimirGrilla(Node *first)
{
  Node *aux = first;
  Node *aux2 = aux->next;

  while (aux2 != first)
  {
    printf("%b -> ", aux->dato);
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");
  aux2 = aux->next;
  while (aux2 != first)
  {
    printf("%d -> ", aux->dato);
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");
}
```

`imprimirGrillaM`

funcs_1.c — Imprime marcando entre paréntesis los nodos con máscara 3.

```
void imprimirGrillaM(Node *first)
{
  Node *aux = first;
  Node *aux2 = aux->next;

  printf("\n");
  while (aux2 != first)
  {
    if ((aux->dato & 3) == 3)
    {
      printf("(%d) -> ", aux->dato);
    }
    else
    {
      printf("%d -> ", aux->dato);
    }
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");
}
```

limpiarGrilla

funcs_1.c — Quita los nodos marcados en posición par y los guarda en un vector.

```
void limpiarGrilla(Node *first)
{
  Node *aux = first;
  Node *aux2 = aux->next;
  Node *aux3 = aux->next;
  int count = 1, i = 0;
  Node *punt;

  punt = (Node *)malloc(sizeof(Node));
  if (punt == NULL)
  {
    perror("No se pudo guardar el espacio");
    return;
  }

  printf("\n");

  while (aux2 != first)
  {
    if (((aux->dato & 3) == 3) && ((count % 2 == 0)))
    {
      i++;
      punt = (Node *)realloc(punt, sizeof(Node) * (i)); // no sumo count para actualizar cuales son los pares y cuales n
      punt[i - 1] = *aux;
      aux3->next = aux->next;
    }
    else
    {
      printf("%d -> ", aux->dato);
      count++;
    }
    aux3 = aux;
    aux = aux->next;
    aux2 = aux;
  }
  printf("NULL \n");

  printf("\n");

  for (int x = 0; x < i; x++)
  {
    printf("%d -> ", (punt + x)->dato);
  }
  printf("NULL \n");

  for (int x = 0; x < i; x++)
  {
    free(&punt[x]);
  }

  printf("\n");
}
```

freeLista

funcs_1.c — Libera la lista circular.

```
void freeLista(Node *first)
{
  Node *aux = first;
  Node *aux2 = aux->next;

  while (aux2 != first)
  {
    aux2 = aux->next;
    free(aux);
    aux = aux2;
  }
  printf("Memoria Liberada \n");
}
```

6. Grilla tipo buscaminas (qnode_t, Ejercicio 2)

Grilla de qnode_t (campo valor) donde una celda es mina si (valor ^ (1 << 4)) % 7 == 0. Incluye impresión, mapa de minas, el chequeo de una celda y el conteo de minas vecinas (4 costados + 4 diagonales).

`imp_grid`

Ejercicio 2 (agregado) — Imprime la grilla con separadores.

```
void imp_grid(qnode_t *primerNodo)
{
  qnode_t *aux = primerNodo;
  qnode_t *aux2 = primerNodo;
  while (aux != NULL)
  {

    while (aux != NULL)
    {
      printf(" %d |", aux->valor);
      aux = aux->right;
    }
    aux2 = aux2->down;
    aux = aux2;
    printf("\n");
  }
  printf("\n-------------------------\n");
  return;
}
```

`flag`

Ejercicio 2 (agregado) — Imprime X en las minas y ° en el resto.

```
void flag(qnode_t *primerNodo)
{
  qnode_t *aux = primerNodo;
  qnode_t *aux2 = primerNodo;
  int mask;
  while (aux != NULL)
  {

    while (aux != NULL)
    {

      mask = (aux->valor) ^ (1 << 4);

      if (mask % 7 == 0)
      {
        printf(" X |");
      }
      else
      {
        printf(" ° |");
      }
      aux = aux->right;
    }
    aux2 = aux2->down;
    aux = aux2;
    printf("\n");
  }
  printf("\n-------------------------\n");

  return;
}
```

`cumple`

Ejercicio 2 (agregado) — Devuelve 1 si el nodo es mina (maneja NULL).

```
int cumple(qnode_t *n)
{
  if (n == NULL)
  {
    return 0;
  }
  return ((n->valor ^ (1 << 4)) % 7 == 0);
}
```

mine

Ejercicio 2 (agregado) — Dada una posición (x, y): MINA, BANDERA o cantidad de minas vecinas.

```
void mine(qnode_t *primerNodo, int xx, int y)
{
  qnode_t *aux = primerNodo;

  int mask;

  for (int i = 0; i < y - 1; i++)
  {
    if (aux->down == NULL)
    {
      break;
    }
    aux = aux->down;
  }
  for (int j = 0; j < xx - 1; j++)
  {
    if (aux->right == NULL) // CAMBIO 1: chequeo antes y un solo avance
    {
      break;
    }
    aux = aux->right;
  }

  mask = (aux->valor) ^ (1 << 4);
  int mina = 0;

  if (mask % 7 == 0)
  {
    printf(" MINA \n");
  }
  else
  {
    // Costados   (CAMBIO 2: uso cumple)
    if (cumple(aux->up))
    {
      mina++;
    }
    if (cumple(aux->down))
    {
      mina++;
    }
    if (cumple(aux->right))
    {
      mina++;
    }
    if (cumple(aux->left))
    {
      mina++;
    }
    // Diagonales   (CAMBIO 3: cumple ya maneja NULL)
    if (aux->up != NULL)
    {
      if (cumple(aux->up->left))
      {
        mina++;
      }
      if (cumple(aux->up->right))
      {
        mina++;
      }
    }
    if (aux->down != NULL)
    {
      if (cumple(aux->down->left))
      {
        mina++;
      }
      if (cumple(aux->down->right))
      {
        mina++;
      }
    }
    if (mina == 0)
    {
      printf(" BANDERA \n");
```

```
    }
    else
    {
      printf(" Cantidad de minas =%d \n", mina);
    }
  }
  return;
}
```

7. Árbol binario de búsqueda

Árbol con contador de repeticiones por nodo.

`crearNodo`

funciones_0121e7.c — Crea un NodoArbol con cantidad = 1.

```
NodoArbol *crearNodo(int num)
{
    NodoArbol *nodo = (NodoArbol *)malloc(sizeof(NodoArbol));

    if (nodo == NULL)
    {
        printf("No se pudo crear el nodo");
        return NULL;
    }

    nodo->dato = num;
    nodo->cantidad = 1;
    nodo->izq = NULL;
    nodo->der = NULL;

    return nodo;
}
```

`insOrdenado`

funciones_0121e7.c — Inserta ordenado; si ya existe incrementa cantidad.

```
void insOrdenado(NodoArbol **first, int num)
{
    NodoArbol *nuevoNodo = crearNodo(num);

    if (*first == NULL)
    {
        *first = nuevoNodo;
        return;
    }

    NodoArbol *temp = *first;

    while (1)
    {

        if (temp->dato > num)
        {
            printf("\nYendo a la izquierda de : %d |", temp->dato);
            if (temp->izq == NULL)
            {
                temp->izq = nuevoNodo;
                printf("\nValor final  %d |\n\n\n\n", nuevoNodo->dato);
                return;
            }

            temp = temp->izq;
        }
        else if (temp->dato < num)
        {
            printf("\nYendo a la derecha de : %d |", temp->dato);
            if (temp->der == NULL)
            {
                temp->der = nuevoNodo;
                printf("\nValor final  %d |\n\n\n\n", nuevoNodo->dato);
                return;
            }

            temp = temp->der;
        }
        else if (temp->dato == num)
        {
            temp->cantidad = temp->cantidad + 1;
            printf("\n Ahora la cantidad de %d es %d \n\n", num, temp->cantidad);
            return;
        }
    }
}
```

printTree

funciones_0121e7.c — Imprime el árbol con sangría por nivel.

```
void printTree(NodoArbol *n, int level)
{
    if (n == NULL)
    {
        return;
    }

    printf("\n");

    for (int i = 0; i < level; i++)
    {
        printf("    ");
    }
    printf("└── %d - Q: %d ", n->dato, n->cantidad);
    printTree(n->der, level + 1);
    printTree(n->izq, level + 1);
}
```